

Pontificia Universidad Javeriana

Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

Introducción a Sistemas Distribuidos

Decisiones de Diseño Gestión Inteligente de Tráfico Urbano

Integrantes:

Daniel Felipe Ramirez Vargas
Ana Sofia Arboleda
Samuel Pico
Guillermo Andrés Aponte Cárdenas

Docente: John Jairo Corredor

Fecha: 18 de mayo de 2026

Índice

1. Resumen del Diseño Acordado	4
2. Decisiones Base del Proyecto	4
2.1. Supuestos Generales	4
2.2. Parámetros Configurables	4
2.3. Documentación del Archivo de Configuración	5
2.4. Organización del Código Compartido	6
2.4.1. Formato de Logs Operativos	7
2.5. Organización del Repositorio por Computador	8
3. Modelo de la Ciudad	8
3.1. Cuadrícula y Grafo Dirigido	8
3.2. Intersecciones	9
3.3. Vías	9
3.4. Nodos de Borde	9
4. Sensores y Estado del Tráfico	10
4.1. Sensores por Arista	10
4.2. Cámara	10
4.3. Espira Inductiva	10
4.4. GPS	10
4.5. Relación entre Sensores y Aristas	11
4.6. Clasificación Cualitativa de Notas	11
5. Lógica de Analítica y Semáforos	12
5.1. Score por Vía	12
5.2. Pesos y Normalización	12
5.3. Comparación entre Direcciones en Conflicto	13
5.4. Temporización Semafórica	13
5.5. Control Manual	14
6. Vehículos y Simulación	15
6.1. PC0 como Dueño del Mapa, los Vehículos y el Reloj Lógico	15
6.2. Modelo de Vehículo	15
6.3. Movimiento dentro de la Cuadrícula	15
6.4. Registro Operativo de Vehículos	16
6.5. Ambulancia	16
7. Distribución por Computadores	16
7.1. PC0	16
7.1.1. Generación de Vehículos Normales	17
7.2. PC1	17
7.3. PC2	18
7.4. PC3	18

8. Persistencia y Tiempo de Simulación	20
8.1. Base de Datos en PC3	20
8.2. Réplica en PC2	20
8.3. Histórico Diario en PC0	20
8.4. Comportamiento de las Bases entre Ejecuciones	21
8.5. Reloj de Simulación	21
8.6. Overhead de Ejecución Real	22
9. Interacción entre Componentes	22
9.1. Flujo General del Sistema	22
9.2. Snapshot Operativa	23
9.3. Creación de Ambulancias	23
9.4. Priorización Manual de Semáforos	24
10. Inicialización del Sistema	24
10.1. Configuración Inicial	24
10.2. Orden de Arranque	25
10.3. Prioridad de Implementación	25
11. Fallos y Continuidad Operativa	26
11.1. Caída de PC3	26
11.2. Continuidad con PC2	26
11.3. Backend Principal y Backend de Respaldo	26
11.4. Limitaciones Durante la Falla	27
12. Comparación de Rendimiento del Broker	27
12.1. Versión Base	27
12.2. Versión Modificada con Hilos	27
13. Diagramas de Diseño	28
13.1. Diagrama de Clases	28
13.2. Diagrama de Componentes	29
13.3. Diagrama de Despliegue	30
13.4. Diagrama de Secuencia	31
14. Protocolo de Pruebas	32
14.1. Objetivo	32
14.2. Alcance	32
14.3. Supuestos	33
14.4. Marco de Pruebas por Modelos del Sistema	33
14.4.1. Modelo Arquitectónico	33
14.4.2. Modelo de Interacción	33
14.4.3. Modelo de Fallos	34
14.4.4. Modelo de Seguridad	34
14.5. Matriz de Casos de Prueba	34
14.5.1. Pruebas Funcionales	34
14.5.2. Pruebas de Integración	36
14.5.3. Pruebas de Fallos	37
14.5.4. Pruebas de Seguridad e Integridad	38

14.5.5. Pruebas de Desempeño	38
14.6. Procedimiento de Ejecución y Repetición Experimental	40
14.6.1. Preparación del entorno	40
14.6.2. Ejecución de una corrida	40
14.6.3. Repetición experimental	41
14.6.4. Configuración de escenarios	41
14.7. Tratamiento de Fallas: Caída de PC3 y Continuidad con Réplica PC2 .	41
14.7.1. Protocolo de prueba de caída	41
14.7.2. Criterios de éxito	42
14.8. Riesgos a la Validez de Resultados	42
14.8.1. Amenazas a la validez interna	42
14.8.2. Amenazas a la validez externa	42
14.8.3. Amenazas a la validez de constructo	43
14.9. Resultados Esperados y Formato de Reporte	43
14.10 Medidas de rendimiento	43

1. Resumen del Diseño Acordado

El sistema se organiza sobre cuatro computadores (PC0, PC1, PC2 y PC3) que se comunican con ZeroMQ. La ciudad se representa como una cuadrícula $N \times M$, pero internamente se maneja como un grafo dirigido donde las intersecciones son nodos y las vías son aristas con datos de tráfico. Sobre las aristas o accesos de entrada se ubican sensores lógicos de tipo cámara, espira inductiva y GPS. Sus eventos alimentan un *score* por vía de entrada antes del semáforo (véase la sección 5.1), y ese valor se usa para decidir cuál conflicto del cruce debe recibir prioridad.

En la implementación actual, **PC0 es el dueño real del mapa, de los vehículos y del reloj lógico de simulación**. PC0 genera el estado del sistema por ticks, aplica los comandos semafóricos que recibe desde PC2 y publica *las snapshots operativas* (véase la sección 9.2) para que el resto del sistema observe el mismo instante lógico. PC1 aloja los sensores y el broker ZeroMQ; PC2 ejecuta la analítica, el control semafórico y la réplica operativa; PC3 concentra la base de datos principal, el backend primario de consulta y las solicitudes activas de usuario como ambulancias o control manual.

La estrategia de resiliencia se enfoca en la caída de PC3. En ese escenario, el núcleo PC0-PC1-PC2 sigue funcionando y PC2 sostiene únicamente continuidad mínima y consulta de estado actual; no asume operaciones activas de usuario. El histórico amplio del día simulado se concentra en PC0, mientras que PC2 y PC3 se enfocan en el estado operativo presente. También se plantea un experimento de rendimiento para comparar una versión base del broker con otra mejorada con hilos.

Todo el diseño es parametrizable: el tamaño de la cuadrícula, las intersecciones activas, las frecuencias de sensores, los pesos de la analítica, los tiempos semafóricos, el reloj lógico y las reglas de tráfico se definen en archivos de configuración compartidos.

2. Decisiones Base del Proyecto

2.1. Supuestos Generales

- Se asume exactamente **un sensor lógico de cada tipo** (cámara, espira inductiva y GPS) por cada arista o acceso observado en el sistema.
- Todas las vías son de sentido único.
- El cambio de semáforo ocurre únicamente entre verde y rojo, sin fase amarilla.
- Todo el sistema se diseña de forma **parametrizada**, de modo que la escala pueda aumentarse o reducirse sin modificar la lógica central.
- La cantidad total de sensores se deriva de las aristas o accesos observados: si hay k aristas instrumentadas, existirán $3k$ sensores lógicos.

2.2. Parámetros Configurables

Los siguientes parámetros deben poder ajustarse con archivos de configuración, sin recompilar el sistema:

- Tamaño de la cuadrícula de la ciudad ($N \times M$).

- Conjunto de intersecciones activas dentro de la cuadrícula.
- Frecuencia de generación de eventos por tipo de sensor.
- Endpoints ZeroMQ (IP o nombre de host y puerto) para soportar pruebas locales o distribución real en red.
- Tiempos base de alternancia semafórica (por defecto, 15 segundos en condiciones normales).
- Reglas y umbrales que definen los estados de tráfico (normal, congestión, priorización).
- Pesos de ponderación de cada sensor para el cálculo del *score* (véase la sección 5.1).
- Parámetros del reloj de simulación (hora de inicio, hora de fin, factor de aceleración).
- Parámetros de generación de vehículos en PC0 (tasa de ingreso, velocidades, etc.).

2.3. Documentación del Archivo de Configuración

El archivo principal de configuración es `config/system_config.json`. Como el proyecto lo carga con `json.load`, no es posible usar comentarios nativos de JSON sin romper el parseo. Por esa razón se adoptó una convención explícita: incluir claves `_comentario` o `_comentarios` dentro del propio archivo para documentar el propósito de cada bloque y de cada parámetro sin afectar la ejecución.

En la implementación actual, ese archivo agrupa sus parámetros en seis bloques principales:

- **ciudad**: define la geometría de la cuadrícula, el rango de longitud de las vías y la semilla usada para construir un mapa reproducible.
- **sensores**: define qué tipos de sensores están activos y con qué frecuencia publica PC1 sus eventos derivados de *la snapshot* (véase la sección 9.2).
- **analitica**: define los pesos usados para combinar cámara, espira y GPS en el *score* por vía (véase la sección 5.1).
- **simulacion**: define la duración del tick real, el tiempo simulado por tick, la frecuencia de envío de las snapshots (véase la sección 9.2), la tasa de generación de vehículos, el mínimo y máximo de vehículos por vía de entrada activa en cada tick, el rango de velocidades iniciales y la velocidad por defecto de ambulancias.
- **frontend**: define el host y el puerto inicial de la interfaz web local de PC3.
- **zmq**: define todos los endpoints de comunicación entre PC0, PC1, PC2 y PC3, incluyendo ingestas de bases de datos, broker, snapshots (véase la sección 9.2), backend principal, backend de respaldo, control manual y solicitudes de ambulancia.

Dentro del bloque `simulacion` también quedan parametrizadas de forma explícita `hora_inicio_simulada`, `hora_fin_simulada` y `modo_bucle_infinito`, para que el reloj lógico del sistema no dependa solo de lo escrito en el informe sino de valores reales del archivo de configuración. Cuando `modo_bucle_infinito=true`, la simulación ignora ese rango, arranca a las 00:00 y, al pasar de 23:59, vuelve a 00:00 sin detenerse. En ese mismo bloque también se controla `intervalo_snapshot_ticks`, que define cada cuántos ticks PC0 propaga la foto operativa del mapa al resto del sistema.

Esta convención tiene dos ventajas importantes para el proyecto, el archivo sigue siendo ejecutable tal como está y, al mismo tiempo, queda autoexplicado para cualquier integrante del grupo (o para el mismo ingeniero Corredor) que necesite modificar parámetros sin tener que rastrear primero el código fuente.

2.4. Organización del Código Compartido

El directorio `common` centraliza el código que debe ser reutilizado por varios procesos distribuidos del sistema. Su objetivo es reducir duplicación, evitar inconsistencias entre nodos y mantener una única fuente de verdad para contratos, modelos y utilidades compartidas.

La organización adoptada es:

- `common/mensajes`: contratos de comunicación, por ejemplo eventos, comandos, solicitudes de ambulancia y *las snapshots operativas* (véase la sección 9.2).
- `common/modelos`: entidades del dominio y del mundo simulado, por ejemplo grafo de ciudad, intersecciones, vías, vehículos y motor de simulación.
- `common/utilidades`: funciones auxiliares reutilizables, como configuración, logs, normalización de sensores, persistencia SQLite y ayudas de mensajería ZeroMQ.

Es importante aclarar que ubicar una pieza en `common` no le da autoridad operativa a cualquier computador. La autoridad sigue definida por la arquitectura: por ejemplo, PC0 es el único dueño del mapa y de los vehículos, aunque los contratos de mensajes o parte de la lógica compartida vivan en código común.

Dentro de esta organización, la carpeta `common/utilidades` cumple un papel especialmente importante porque concentra tareas transversales que deben comportarse igual en todos los computadores. Entre ellas se encuentran:

- carga de configuración compartida;
- generación de logs con formato uniforme;
- normalización y clasificación de sensores;
- persistencia SQLite compartida;
- y ayudas de mensajería ZeroMQ reutilizables.

Esta separación permite que la lógica principal del sistema quede en los modelos y en los procesos específicos de cada PC, mientras que `common/utilidades` funciona como una caja de herramientas compartida que evita reimplementar operaciones auxiliares una y otra vez.

En la implementación actual, el contenido principal de `common` puede resumirse así:

- `common/mensajes/eventos.py`: eventos de sensores emitidos por PC1, incluyendo la vía observada y el `tick_origen`.
- `common/mensajes/comandos.py`: comandos semafóricos emitidos por PC2 hacia PC0.
- `common/mensajes/estado_operativo.py`: contrato de *la snapshot operativa* (véase la sección 9.2) que representa la foto actual del sistema.
- `common/mensajes/ambulancias.py`: solicitud compartida para crear ambulancias en PC0.
- `common/modelos/trafico.py`: grafo de la ciudad, intersecciones, nodos de borde y vías.
- `common/modelos/vehiculos.py`: entidades vehiculares del sistema.
- `common/modelos/simulacion.py`: motor de simulación por ticks y reglas de movimiento.
- `common/utilidades/configuracion.py`: carga y acceso a configuración compartida.
- `common/utilidades/logs.py`: formato uniforme de logs.
- `common/utilidades/normalizacion_sensores.py`: fórmulas y clasificación de sensores.
- `common/utilidades/persistencia_sqlite.py`: persistencia compartida en SQLite.
- `common/utilidades/mensajeria_zmq.py`: ayudas ZeroMQ de mejor esfuerzo.

2.4.1. Formato de Logs Operativos

Se adopta un formato uniforme de logs para todos los procesos del sistema:

```
[real=HH:MM:SS] [sim=HH:MM] [NOMBRE_PROCESO]mensaje
```

La hora real corresponde al reloj del computador donde corre el proceso. La hora simulada se calcula a partir de `simulacion.hora_inicio_simulada`, `simulacion.minutos_simulados_por_tick` y el `tick` asociado al evento, `snapshot` o `comando`. Cuando un proceso todavía no tiene contexto de simulación, el campo se imprime como `sim=-`.

Esta decisión permite leer en una misma línea el momento real de ejecución y el instante lógico del mundo simulado, lo cual facilita depuración, sustentación y medición de latencias.

2.5. Organización del Repositorio por Computador

Además de `common`, el repositorio se estructura por responsabilidades de cada computador:

- `PC0/simulation`: simulación autoritativa del mapa, del tick y de los vehículos.
- `PC0/historic_db`: persistencia histórica amplia del día simulado.
- `PC1/sensors`: sensores lógicos que leen las snapshots (véase la sección 9.2) y publican eventos.
- `PC1/broker`: broker ZeroMQ base.
- `PC2/analytics`: analítica y decisión semafórica por tick.
- `PC2/traffic_ctrl`: envío de comandos hacia PC0.
- `PC2/replica_db`: réplica operativa del estado actual.
- `PC2/backend_respaldo`: backend de continuidad limitado a salud y estado presente.
- `PC3/main_db`: persistencia principal y resincronización.
- `PC3/backend`: backend primario de consulta, ambulancias y control manual.
- `PC3/frontend`: servidor HTTP local de la visualización de PC3 y archivos estáticos separados (`index.html`, `styles.css`, `app.js`).

Esta distribución busca que el repositorio exprese de manera visible la arquitectura lógica del sistema. En otras palabras, no solo separa archivos por conveniencia, sino también por responsabilidad, es decir, dónde vive la simulación, dónde se sensa, dónde se analiza, dónde se respalda el estado y dónde se expone el backend principal.

3. Modelo de la Ciudad

3.1. Cuadrícula y Grafo Dirigido

La ciudad se describe como una cuadrícula $N \times M$ (filas por letras y columnas por números). Internamente, como **decisión del grupo**, se modela como un **grafo dirigido** sobre esa cuadrícula, es decir, las intersecciones son nodos y las vías de sentido único son aristas dirigidas con atributos. Este modelo permite representar la dirección del flujo, la congestión y el estado de cada vía.

En la visualización se puede mostrar información sobre las vías como si hubiera puntos intermedios, pero en la lógica del sistema cada vía sigue siendo una arista con atributos.

3.2. Intersecciones

Cada intersección es un nodo del grafo con los siguientes atributos:

- **Identificador único:** por ejemplo INT-C5 (fila C, columna 5).
- **Coordenada** (*fila, columna*).
- **Sensores relacionados:** las aristas o accesos conectados a la intersección cuentan con sensores lógicos de cámara, espira inductiva y GPS.
- **Dos semáforos lógicos:** eje horizontal y eje vertical.
- **Fase activa:** HORIZONTAL o VERTICAL; nunca hay dos fases en verde simultáneamente.

3.3. Vías

Cada vía se modela como una arista dirigida del grafo con los siguientes atributos. Como todas las vías son unidireccionales, cada arista representa **un solo sentido de circulación**:

- Intersección origen.
- Intersección destino (o nodo de borde destino).
- Longitud o costo de la vía.
- Dirección cardinal del movimiento (NORTE, SUR, ESTE u OESTE).
- Vehículos en circulación dentro de la arista.
- Vehículos en espera (detenidos por semáforo en rojo al final de la arista).
- Velocidad promedio observada.
- *Score* de congestión (véase la sección 5.1), como valor normalizado en $[0, 1]$.
- Estado de congestión derivado del *score* (véase la sección 5.1): tráfico normal, congestión o priorización.

3.4. Nodos de Borde

Los bordes de la cuadrícula se modelan mediante **nodos de borde**, que permiten representar tanto la entrada como la salida de vehículos de la ciudad (o del fragmento simulado). Sus atributos son:

- Identificador único.
- Intersección del borde a la que está conectado.
- Lado del borde: NORTE, SUR, ESTE u OESTE.

Estos nodos habilitan el ingreso de vehículos generados por PC0 y el egreso cuando un vehículo alcanza el borde de la ciudad. La convención adoptada usa números de columna para norte/sur (por ejemplo BORDE-S-1) y letras de fila para oeste/este (por ejemplo BORDE-O-A).

4. Sensores y Estado del Tráfico

4.1. Sensores por Arista

Cada arista o acceso instrumentado cuenta con sensores lógicos de cada tipo: cámara, espira inductiva y GPS. En esta solución, los sensores no se entienden como dispositivos ubicados físicamente en el punto de la intersección, sino como **observadores de aristas o accesos**. En la práctica, funcionan como **getters** del estado de la arista: consultan de forma periódica los atributos del tramo observado y procesan esos datos para producir las estadísticas y eventos que necesita la analítica. Como la vía tiene un único sentido, el sensor solo mide el flujo que viene en ese sentido hacia el semáforo.

4.2. Cámara

La cámara mide la acumulación o cola de vehículos y aporta una señal de **ocupación inmediata**. Sus datos salen de leer la cantidad de vehículos en espera al final de la arista observada (`EVENTO_LONGITUD_COLA`, L_q), es decir, los vehículos que ya completaron la longitud de la vía y se encuentran detenidos antes de la intersección.

La nota de prioridad aportada por la cámara se calcula como:

$$n_c = \min\left(\frac{\text{vehículos en espera}}{10}, 1\right) \quad (1)$$

Esto implica que 1 vehículo en espera aporta 0.1, 5 vehículos aportan 0.5 y 10 o más aportan 1.0.

4.3. Espira Inductiva

La espira inductiva mide la cantidad de vehículos que están en tránsito dentro de la arista antes de llegar a la intersección y aporta una señal de **presión de tráfico** (`EVENTO_CONTEO_VEHICULAR`, C_v). En esta versión del diseño, la espira **no mide la cola**; solo mide vehículos que todavía recorren la arista.

La nota de prioridad aportada por la espira se calcula como:

$$n_e = \min\left(\frac{\text{vehículos en tránsito}}{10}, 1\right) \quad (2)$$

Esto implica que 1 vehículo en tránsito aporta 0.1, 5 vehículos aportan 0.5 y 10 o más aportan 1.0.

4.4. GPS

El sensor GPS mide la velocidad promedio de los vehículos **en tránsito** en una arista y aporta una señal complementaria de **fluidez o lentitud** (`EVENTO_DENSIDAD_DE_TRAFICO`, D_t). Como decisión del grupo, el GPS **no se usa como criterio único** para cambiar semáforos, sino como un componente ponderado dentro del *score* total (véase la sección 5.1).

Si no hay vehículos en tránsito, el promedio de velocidad es 0 y la nota del GPS también es 0. Si sí hay vehículos circulando, el promedio queda entre 10 y 50 porque cada vehículo mantiene una velocidad propia dentro de ese rango mientras se mueve por la vía.

La nota del GPS se calcula por tramos:

- Si $x = 0$, entonces $n_g = 0$.
- Si $10 \leq x \leq 50$, entonces:

$$n_g = 1 - 0,9 \cdot \frac{x - 10}{40} \quad (3)$$

Con esta normalización, 10 se mapea a 1.0 y 50 se mapea a 0.1. En otras palabras, a menor velocidad promedio de tránsito, mayor prioridad aporta el GPS.

4.5. Relación entre Sensores y Aristas

Los sensores simulados en PC1 están definidos sobre aristas o accesos y consultan el estado de los vehículos mantenido por PC0 para generar eventos realistas. En otras palabras, cada sensor actúa como un getter especializado del estado de una vía: toma atributos del tramo observado, los procesa según su tipo y publica la estadística resultante. Así, la simulación en PC0 alimenta de forma directa el flujo de datos del resto del sistema.

En la implementación actual, los eventos se emiten **por arista instrumentada** y cada evento identifica explícitamente la vía observada, la intersección destino y el `tick_origen` de la simulación. Esto permite que PC2 agrupe lecturas coherentes del mismo instante lógico antes de calcular una decisión semafórica.

4.6. Clasificación Cualitativa de Notas

Las notas normalizadas de los sensores se interpretan también cualitativamente con la misma escala. Esta clasificación se aplica de manera uniforme a las notas parciales de cámara, espira inductiva y GPS, de modo que los valores numéricos no solo sirvan para calcular el *score* (véase la sección 5.1), sino también para comunicar de forma más clara el nivel de tráfico observado en cada vía:

- **Bajo:** $0,0 \leq x < 0,4$
- **Moderado:** $0,4 \leq x \leq 0,7$
- **Intenso:** $0,7 < x \leq 1,0$

Esta escala facilita la interpretación humana del sistema y también sirve como base natural para una futura visualización del mapa, donde una categoría cualitativa resulta más fácil de mostrar y discutir que una nota continua aislada.

5. Lógica de Analítica y Semáforos

5.1. Score por Vía

La lógica de control semafórico se basa en una **calificación de estado** (*score*) para cada vía de entrada a una intersección. Ese *score* solo describe la carga del tráfico **antes del semáforo** y en el **único sentido** permitido de la vía. Cada sensor aporta al *score* con una ponderación configurable. Entre mayor sea el *score*, mayor será la prioridad para recibir verde. El valor final queda normalizado en $[0, 1]$. Los tres sensores aportan de forma conjunta, sin depender de un solo indicador.

5.2. Pesos y Normalización

Cada sensor produce primero una **nota normalizada** entre 0 y 1, de acuerdo con el estado observado en la arista. Después, esas tres notas se combinan con pesos configurables para obtener un *score* final, también entre 0 y 1.

Si se denotan las notas de cámara, espira y GPS como n_c , n_e y n_g , entonces:

$$n_c, n_e, n_g \in [0, 1] \quad (4)$$

$$w_c, w_e, w_g \in [0, 1] \quad (5)$$

$$w_c + w_e + w_g = 1 \quad (6)$$

Para esta versión del diseño, se fijan los siguientes pesos:

- $w_c = 0,50$: peso de la **cámara**. Representa la importancia de la cola o acumulación de vehículos en la vía.
- $w_e = 0,35$: peso de la **espira inductiva**. Representa la importancia del flujo de vehículos que pasan por la vía.
- $w_g = 0,15$: peso del **GPS**. Representa la importancia de la velocidad promedio como señal complementaria de fluidez o lentitud.

Con estos valores, el *score* de la vía se calcula como:

$$\text{score}_{\text{via}} = w_c n_c + w_e n_e + w_g n_g \quad (7)$$

Como los pesos suman 1 y cada nota está en $[0, 1]$, el resultado final también queda en el rango $[0, 1]$. En esta distribución, la cámara pesa más porque la cola de vehículos es la señal más importante para decidir prioridad; la espira tiene un peso intermedio porque muestra presión de flujo; y el GPS pesa menos porque solo complementa la lectura de congestión. Estos valores pueden ajustarse más adelante si el grupo decide recalibrar el sistema.

Para mantener coherencia entre analítica, persistencia y visualización, PC0 calcula y publica en cada snapshot operativo el mismo *score* ponderado que usa PC2: cámara por vehículos en espera, espira por vehículos en circulación y GPS por velocidad promedio. El estado textual de la vía se deriva de ese *score*: **BAJA** para valores menores a 0,40, **NORMAL** entre 0,40 y 0,70, y **ALTA** por encima de 0,70.

5.3. Comparación entre Direcciones en Conflicto

Cada intersección se controla comparando las vías de entrada que están en conflicto antes del semáforo. El *score* no se calcula por eje como valor base, sino por cada vía de entrada. Por ejemplo, una intersección puede tener:

- una vía que llega desde el norte con su propio *score*,
- una vía que llega desde el sur con su propio *score*,
- una vía que llega desde el este con su propio *score*,
- una vía que llega desde el oeste con su propio *score*.

Con esos valores, el servicio de analítica en PC2 compara las direcciones que compiten por el paso en el cruce. La dirección con mayor *score* recibe prioridad. Si los puntajes son parecidos, se mantiene la alternancia normal o se aplica una regla de desempate configurable.

En la práctica, el servicio de analítica en PC2 revisa de manera continua los conflictos de todas las intersecciones del grafo. Para cada nodo identifica las vías de entrada que compiten por el paso, consulta sus *scores* actuales y decide cuál debe recibir prioridad semafórica en ese momento. Como el control final se expresa por ejes lógicos (HORIZONTAL y VERTICAL), la implementación obtiene el *score* por eje como el promedio de los *scores* de las vías de entrada asociadas a ese eje.

5.4. Temporización Semafórica

La duración del verde se ajusta según la diferencia entre los *scores* de las direcciones que están en conflicto. La implementación usa un **contador cíclico fijo de 30 ticks** por intersección. Ese contador avanza de 1 a 30, vuelve a 1 y **no se reinicia** cuando llega una orden nueva desde PC2.

La orden automática ya no significa “cambie inmediatamente y empiece a contar otra vez”. En cambio, significa: con el contador actual, reserve esta cantidad de ticks del ciclo para el eje prioritario y el resto para el eje opuesto. Por eso, si una nueva lectura cambia los *scores*, la partición del ciclo puede moverse en tiempo real sin borrar el avance del contador. Se define:

$$\text{gap} = |\text{score}_1 - \text{score}_2| \in [0, 1]$$

- Si $\text{gap} = 0$: no existe diferencia de prioridad entre los dos ejes comparados; cada uno recibe **15 ticks**, completando un ciclo total de 30 ticks.
- Si $0 < \text{gap} < 1$: el eje con mayor *score* recibe una porción mayor del ciclo total de 30 ticks. Entre mayor sea el *gap*, mayor será su ventana de verde y menor la ventana del eje opuesto.
- Si $\text{gap} = 1$: significa que un eje tiene prioridad máxima frente al otro. En ese caso, el eje ganador recibe los **30 ticks completos** del ciclo.

De forma general, si $\text{score}_{\text{max}}$ es el mayor de los dos *scores* comparados, la ventana del eje prioritario puede expresarse como:

$$T_{\text{verde}} = 15 + 15 \cdot \text{gap} \quad (8)$$

De este modo:

- con $\text{gap} = 0$, $T_{\text{verde}} = 15$ ticks;
- con $\text{gap} = 1$, $T_{\text{verde}} = 30$ ticks.

El tiempo restante del ciclo queda asignado al eje opuesto:

$$T_{\text{opuesto}} = 30 - T_{\text{verde}} \quad (9)$$

Adicionalmente, la implementación aplica una excepción operativa: si y solo si un eje tiene $\text{score}=0.0$ y el otro tiene $\text{score}>0.0$, el eje con carga recibe los 30 ticks completos y el eje vacío recibe 0 ticks. Esta regla evita dejar esperando vehículos cuando el conflicto opuesto no tiene ningún carro.

Operativamente, el eje opuesto ocupa el primer tramo del contador y el eje prioritario ocupa el tramo final. Por ejemplo, si **VERTICAL** gana con $T_{\text{verde}} = 23$ y $T_{\text{opuesto}} = 7$, entonces el contador 1–7 da verde a **HORIZONTAL** y el contador 8–30 da verde a **VERTICAL**. Si después llega otra orden, PC0 conserva el contador actual y recalcula qué eje corresponde a ese punto del ciclo. La discretización se hace a ticks enteros; por ejemplo, un valor teórico de 22.5 ticks se materializa como 23 ticks para el eje prioritario y 7 para el opuesto.

Además de esta temporización, la implementación actual aplica dos reglas operativas importantes:

- **Deduplicación de comandos:** si para una intersección la fase y los tiempos calculados no cambian respecto a la última orden emitida, el sistema no reenvía exactamente el mismo comando.
- **Una decisión por intersección y por tick:** aunque los eventos lleguen secuencialmente, PC2 agrupa las mediciones del mismo `tick_origen` y emite como máximo una decisión por intersección en ese instante lógico.

5.5. Control Manual

Desde PC3 el usuario puede forzar manualmente el estado de un semáforo:

- Selecciona una intersección y un eje a priorizar.
- Mientras ese control esté activo, la lógica automática basada en *score* queda suspendida en esa intersección.
- El forzado se mantiene hasta nueva orden; no expira por tiempo.
- Solo una intersección puede permanecer en modo manual a la vez.
- Para cambiar de intersección controlada, primero debe devolverse la anterior a modo automático.

El forzado se expresa en tiempo de simulación y no en tiempo real.

6. Vehículos y Simulación

6.1. PC0 como Dueño del Mapa, los Vehículos y el Reloj Lógico

Como **extensión del grupo de computadores**, se incorpora PC0 para la simulación de vehículos que cambian el estado del tráfico. En la implementación actual, PC0 no es solo un generador de vehículos: es el **dueño real del mapa, del estado de las vías, de los vehículos y del reloj lógico de simulación**. Desde PC0 se generan vehículos que entran a la ciudad por nodos de borde, se aplican los comandos semafóricos que llegan desde PC2 y se calcula el estado que será propagado al resto del sistema.

Además, PC0 publica *las snapshots operativas* (véase la sección 9.2) con el estado actual hacia PC1, para que los sensores lean esa foto del sistema sin duplicar la simulación. PC0 también almacena el histórico completo de un día simulado para análisis posterior y comunica el estado presente a la base principal de PC3 y a la réplica operativa de PC2.

6.2. Modelo de Vehículo

Cada vehículo es una entidad simulada con identificador único. Un vehículo solo puede estar en una arista a la vez. Sus atributos mínimos son:

- Identificador del vehículo.
- Tipo de vehículo (NORMAL o AMBULANCIA).
- Arista actual en la que se encuentra.
- Posición relativa o progreso dentro de la arista.
- Velocidad simulada.
- Dirección actual de movimiento.
- Estado actual (CIRCULANDO, EN_ESPERA o saliendo del sistema).
- *Timestamp* de última actualización (en tiempo de simulación).

6.3. Movimiento dentro de la Cuadrícula

Los vehículos entran al sistema por un nodo de borde y recorren aristas dirigidas entre intersecciones. Al llegar a una intersección, el vehículo decide aleatoriamente entre seguir derecho o tomar la alternativa permitida. No puede moverse en contra del sentido de una vía. Sale del sistema cuando llega a un nodo de borde configurado como egreso.

La presencia y el movimiento de los vehículos cambian el estado de las vías: cada vehículo aporta al conteo de vehículos en circulación dentro de su arista. En cada tick, la fase semafórica vigente se actualiza antes de mover los vehículos. Si un carro alcanzaría una intersección cuyo semáforo no favorece el eje de su vía, el motor lo detiene en el borde de la intersección, lo marca como EN_COLA y no le permite entrar ni cruzar

el nodo. Solo cuando la fase pasa a ser favorable puede seleccionar la siguiente vía y continuar. Estos valores alimentan las mediciones de los sensores simulados.

La velocidad de tránsito se asigna al vehículo cuando se instancia y permanece como atributo propio del vehículo. Cuando el carro logra pasar por una intersección con semáforo en verde y entra a una nueva vía, continúa recorriendo esa nueva arista con la misma velocidad que ya tenía asignada. Cuando un vehículo sale de la ciudad, se elimina del estado activo del motor de simulación.

6.4. Registro Operativo de Vehículos

Para facilitar pruebas sin interfaz gráfica, PC0 registra en logs la creación y la salida de vehículos de la simulación. Este registro operativo no reemplaza la persistencia en base de datos, pero sí ofrece una traza inmediata y legible de lo que está ocurriendo en cada tick del motor de simulación.

En los logs de creación se reportan, entre otros, el identificador, el tipo, la vía inicial, el origen, el destino de la vía, la dirección, la velocidad, el estado, el tick y la hora simulada. Esto permite verificar que la generación de tráfico respeta la configuración del sistema y que los vehículos nacen en los nodos de borde correctos.

En los logs de salida se reporta también el motivo de retiro del vehículo del estado activo, por ejemplo salida efectiva de la ciudad o ausencia de una ruta de continuación válida. De esta forma, cuando un vehículo deja de existir dentro de la simulación, esa eliminación no queda implícita sino registrada de forma explícita.

Como complemento, el resumen de cada tick incluye la hora simulada correspondiente al estado actual del motor de simulación lo cual permite relacionar creación, movimiento y salida de vehículos con el reloj lógico propagado por PC0.

6.5. Ambulancia

Desde PC3 el usuario puede solicitar manualmente una ambulancia en un nodo de borde con *spawn* válido. La ambulancia cuenta como un vehículo más, pero con una representación visual distinta, una velocidad constante configurable y prioridad total sobre la vía que ocupa. La ambulancia sale del sistema al llegar a un nodo de borde.

Operativamente, la ambulancia **la solicita PC3 y la crea PC0**. El backend principal de PC3 construye la solicitud y la envía por ZeroMQ a PC0; PC0 recibe esa solicitud y la inyecta en el motor de simulación como un vehículo de tipo **AMBULANCIA**. Mientras la ambulancia ocupe una vía, esa vía fuerza **score=1.0** en PC0 y PC1 publica notas de sensores 1.0 para esa misma vía, de modo que PC2 también le otorgue prioridad total al eje correspondiente.

7. Distribución por Computadores

7.1. PC0

Extensión propuesta al grupo de computadores. PC0 es el dueño del estado autoritativo de la simulación y mantiene el estado base del tráfico. Sus responsabilidades son:

- Generar vehículos que ingresan a la ciudad por nodos de borde.

- Mantener y actualizar la posición de cada vehículo en el grafo.
- Mantener el estado autoritativo del mapa, las vías y los semáforos dentro de la simulación.
- Instanciar ambulancias cuando PC3 lo solicite por ZeroMQ.
- Recibir desde PC2 los comandos semafóricos automáticos y aplicarlos sobre la simulación.
- Publicar las snapshots operativas del estado actual (véase la sección 9.2) hacia PC1.
- Comunicar el estado de los vehículos en el grafo-mapa a la base de datos principal de PC3, a la réplica de PC2 y a su propia base histórica en PC0.
- Almacenar el historial completo de un día de simulación para estadísticas finales.

PC0 **no** forma parte del mecanismo principal de respaldo cuando ocurre una falla; su almacenamiento histórico es para análisis posterior, no para mantener la operación del sistema.

7.1.1. Generación de Vehículos Normales

La generación normal de vehículos se evalúa por cada vía de entrada desde borde. En cada tick, PC0 recorre esas vías de entrada y aplica `simulacion.probabilidad_generacion_por_via`. Si la probabilidad activa la entrada, el motor crea en esa misma vía una cantidad aleatoria de vehículos entre `simulacion.min_nuevos_por_tick` y `simulacion.max_nuevos_por_tick`.

Con la configuración actual, donde `probabilidad_generacion_por_via=1`, cada vía de entrada genera vehículos en todos los ticks. Si `min_nuevos_por_tick=25` y `max_nuevos_por_tick=30`, entonces cada entrada crea entre 25 y 30 vehículos por tick. En una ciudad de 4×4 hay 8 entradas desde borde, por lo que el rango total esperado por tick queda entre 200 y 240 vehículos normales.

7.2. PC1

PC1 aloja los sensores y el broker. Sus responsabilidades son:

- Ejecutar los sensores simulados (cámara, espira inductiva y GPS) como procesos lógicos asociados a aristas que generan eventos periódicos.
- Recibir las snapshots operativas producidas por PC0 (véase la sección 9.2) y usarlas como fuente de verdad para calcular sus mediciones.
- Publicar eventos mediante PUB/SUB de ZeroMQ, con tópicos diferenciados por tipo de sensor.
- Operar el broker ZeroMQ que recibe los eventos y los reenvía a PC2.

7.3. PC2

PC2 aloja la analítica y el control de semáforos. Sus responsabilidades son:

- Suscribirse a los eventos de sensores vía el broker de PC1.
- Calcular el *score* por vía y determinar la fase semafórica de cada intersección.
- Emitir como máximo un comando por intersección y por tick de simulación, agrupando mediciones del mismo `tick_origen`.
- Ejecutar órdenes de control sobre semáforos e imprimir por pantalla las acciones realizadas.
- Recibir y ejecutar indicaciones de control manual provenientes de PC3.
- Mantener la **réplica de la base de datos**, actualizada de forma asíncrona, para que el sistema pueda seguir operando si PC3 falla.
- Exponer un backend de respaldo limitado a salud y consultas de estado actual, sin crear ambulancias ni aceptar control manual durante el fallo.

7.4. PC3

PC3 aloja el monitoreo, la consulta y la persistencia principal. Sus responsabilidades son:

- Alojar la **base de datos principal**.
- Proveer monitoreo y consulta del estado actual mediante REQ/REP.
- Enviar indicaciones directas al servicio de analítica para forzar cambios semafóricos.
- Exponer el backend primario que el cliente consulta normalmente antes de considerar el respaldo de PC2.
- Visualizar la ciudad como grafo sobre cuadrícula mediante una interfaz web local levantada junto con el backend principal.
- Mantener separada la interfaz web de la lógica del backend: el servidor Python de la interfaz vive en `PC3/frontend/servidor.py`, mientras que HTML, CSS y JavaScript residen en `PC3/frontend/static/`.
- Dibujar nodos de spawn/salida y vías sin texto fijo sobre el grafo; la información completa se muestra al pasar el cursor sobre nodos, vías o vehículos.
- Colorear las vías según el *score* de congestión: verde para carga baja, amarillo para carga moderada y rojo para carga alta.
- Incluir una leyenda de fases semafóricas: azul para fase horizontal y morado para fase vertical.

- Mostrar en el inspector de cada vía un recuadro **SCORE** con fórmula, pesos configurados, qué evalúa cada sensor, notas parciales de cámara/espira/GPS y contribución de cada sensor.
- Aumentar la legibilidad del inspector, la leyenda y los tooltips; en el detalle del **SCORE**, separar en líneas distintas la contribución **nota*peso** y la métrica concreta evaluada por el sensor.
- Incluir un botón **Infodesimulacion** en el panel lateral. Al activarlo, el inspector muestra estadísticas globales del snapshot actual: hora simulada, tick rate, cantidad de intersecciones, vías, vehículos, ambulancias, semáforos por fase, vehículos parados/circulando y vehículos generados/despawneados en el último tick.
- Mover **Infodesimulacion** a un botón flotante sobre el lado izquierdo del mapa. El botón despliega un panel independiente con las estadísticas globales, por lo que el panel derecho ya no incluye el bloque resumido de **Estado**.
- Añadir un botón flotante de **Controlmanual** debajo de **Infodesimulacion**, con validación de intersección existente y tres acciones exclusivas: verde horizontal, verde vertical y automático.
- Añadir un botón flotante de **Ambulancia** en la esquina inferior izquierda, con formulario para construir nodos **BORDE-X-X** válidos y enviar la solicitud al backend principal.
- Mantener los paneles flotantes fuera del área de sus propios botones y evitar re-renderizarlos en cada tick mientras el usuario escribe, para no perder foco en los campos del formulario.
- Reemplazar el mapa por una pantalla de cierre cuando la simulación alcance la hora final configurada, mostrando hora de inicio, hora de fin, tick final y hora real del último snapshot.
- Mostrar en esa pantalla final el tiempo real observado, el tiempo teórico esperado por configuración y el overhead o diferencia entre ambos.
- Mostrar en la barra superior tanto el reloj real de la máquina como el reloj lógico de simulación calculado desde `hora_inicio_simulada`, `tick_actual` y `minutos_simulados_por_tick`.
- Refrescar la interfaz web con la misma cadencia configurada para el tick de PC0 (`simulacion.tick_segundos_reales`), evitando acumular solicitudes si el navegador aún está procesando el refresco anterior.
- Recortar visualmente las aristas para que empiecen y terminen en el borde del nodo, no en su centro, y hacer que la punta de la flecha de dirección termine justo sobre el borde del círculo de la intersección destino.
- Resaltar visualmente los vehículos en espera de semáforo (**EN_COLA**) en color rojo, manteniendo los vehículos en circulación en negro.

- Dibujar los vehículos en circulación sobre el mismo segmento visual recortado de la arista, de borde a borde, para que al llegar al semáforo queden ya en el punto donde esperarían si el estado cambia a `EN_COLA`.
- Dibujar los vehículos en cola fijos por fuera del borde del círculo de la intersección, tangentes al lado desde el que llegan; si hay varios esperando por la misma vía, se distribuyen alrededor de ese borde para evitar superposición directa.
- Si el frontend detecta que un vehículo en circulación está a punto de tocar visualmente una intersección cuyo semáforo no favorece su eje, lo muestra ya como detenido en el borde, evitando que el círculo negro entre al nodo y luego salte hacia atrás.
- Usar nodos de intersección, nodos de spawn/salida y vías con mayor tamaño visual para mejorar la lectura del mapa en tiempo real.
- Permitir al usuario solicitar ambulancias en nodos de borde con *spawn* válido.

8. Persistencia y Tiempo de Simulación

8.1. Base de Datos en PC3

La base de datos principal reside en PC3. Su responsabilidad principal es almacenar el **estado operativo actual** de la ciudad en tiempo real: estado de intersecciones, estado de vías, estado actualizado de vehículos y estado vigente de semáforos.

PC3 **no** se concibe como el repositorio principal de histórico amplio de eventos de sensores ni de comandos semafóricos. Ese histórico se concentra en PC0. La base principal de PC3 se enfoca en el estado presente del sistema y en servir como soporte del backend primario.

8.2. Réplica en PC2

La réplica se encuentra en PC2 y se actualiza de forma asíncrona (PUSH/PULL u otro patrón similar). Su propósito es mantener el **estado operativo actual** de la ciudad y el estado de los vehículos reportado desde PC0, de modo que el sistema pueda seguir funcionando si PC3 falla. PC2 no se plantea como almacén de resultados históricos de largo plazo, sino como respaldo del estado presente.

Si PC3 cae, el núcleo operativo **PC0-PC1-PC2** sigue funcionando. Cuando PC3 vuelve a estar disponible, su base de datos se actualiza nuevamente con el estado que tenía PC2 y, una vez termina esa sincronización, el sistema vuelve a operar usando PC3 como base principal. Los envíos a PC3 se manejan en mejor esfuerzo para no bloquear el ciclo principal si la base principal no está disponible.

8.3. Histórico Diario en PC0

PC0 almacena el historial completo de un día de simulación para análisis posterior y estadísticas finales (extensión del grupo de computadores). En esta base también quedan registrados los eventos de sensores, los comandos semafóricos y el estado histórico de los vehículos a lo largo del día simulado. PC0 **no** se utiliza como nodo de resiliencia operativa; su rol es exclusivamente analítico.

8.4. Comportamiento de las Bases entre Ejecuciones

Si el usuario detiene una ejecución y luego vuelve a arrancar el sistema sin borrar las bases SQLite, no todos los computadores se comportan igual:

- La simulación **no se reanuda** desde base de datos. Cada vez que PC0 arranca, construye un mapa nuevo y un motor nuevo en memoria.
- La base histórica de PC0 **sí acumula** datos entre corridas si no se limpia previamente.
- Las bases de PC2 y PC3 conservan el último estado actual persistido, pero ese estado no gobierna la nueva simulación ya que se reemplaza con las snapshots nuevas (véase la sección 9.2) que emita PC0.
- Mientras no llegue la primera snapshot nueva (véase la sección 9.2), una consulta temprana a PC2 o PC3 todavía podría mostrar estado viejo persistido de la corrida anterior.
- Si PC3 arranca antes de que la nueva simulación emita snapshots (véase la sección 9.2), puede resincronizarse temporalmente con la última snapshot vieja almacenada en PC2; eso se corrige en cuanto PC0 vuelve a emitir estado nuevo.

En consecuencia, el comportamiento entre ejecuciones combina dos efectos distintos: por un lado, el motor activo siempre comienza una simulación nueva en memoria; por otro, las bases pueden conservar rastros de la corrida anterior hasta que llegan nuevas actualizaciones o hasta que el usuario las limpia manualmente.

Esto se vuelve especialmente visible cuando PC3 arranca antes de que exista una nueva corrida activa de PC0. En ese caso, PC3 no se queda esperando indefinidamente a que llegue una snapshot nueva (véase la sección 9.2), sino que intenta recuperar de inmediato el estado actual desde PC2. Como PC2 responde con la última foto operativa que tenga guardada, esa respuesta puede corresponder todavía a la ejecución anterior si PC0 aún no ha empezado a emitir estados nuevos y actualizados.

Por esta razón, si se quiere una prueba completamente limpia y sin mezclar corridas anteriores, deben borrarse previamente las SQLite de PC0, PC2 y PC3. Para facilitar esa operación, el proyecto incluye un script de limpieza dedicado dentro del directorio `scripts`.

8.5. Reloj de Simulación

El sistema comparte un reloj global de simulación que representa un día lógico configurable. El ritmo real no está fijo en el código: se controla con `simulacion.tick_segundos_reales`, mientras que el avance lógico se controla con `simulacion.minutos_simulados_por_tick`. Con la configuración actual de prueba acelerada, `tick_segundos_reales=0.03333` y `minutos_simulados_por_tick=1`, por lo que cada 33 ms reales representan 1 minuto simulado.

En la implementación actual, el reloj lógico se materializa en **PC0**. Cada tick del motor incrementa el tiempo simulado y esa referencia temporal se propaga al resto del sistema por medio de las snapshots operativas (véase la sección 9.2) y del `tick_origin` en los eventos. Todos los eventos (sensores, vehículos, semáforos y persistencia) usan el tiempo de simulación, no el tiempo real.

El rango horario queda también parametrizado en `config/system_config.json` mediante `hora_inicio_simulada` y `hora_fin_simulada`. Adicionalmente, la cadencia de propagación del estado del mapa se controla con `intervalo_snapshot_ticks`. En la implementación actual, cuando `modo_bucle_infinito=false`, PC0 usa esa hora final para detener automáticamente el motor cuando el reloj lógico alcanza el cierre del día simulado. Antes de detenerse, PC0 envía el snapshot del tick final, registra el cierre del día y muestra un mensaje en arte ASCII indicando la finalización de la simulación.

Si `modo_bucle_infinito=true`, la lógica cambia deliberadamente: PC0 ignora `hora_inicio_simulada` y `hora_fin_simulada`, fija el reloj lógico en un ciclo continuo 0 0:00->23:59->00:00, no considera nunca la simulación como finalizada y, por tanto, tampoco dispara la pantalla final de cierre en la interfaz web de PC3.

8.6. Overhead de Ejecución Real

Al finalizar el día simulado, la interfaz web de PC3 contrasta el tiempo real observado contra el tiempo teórico esperado por configuración. El tiempo teórico se calcula como:

$$T_{teorico} = \left\lceil \frac{\text{duracion_simulada_min}}{\text{minutos_simulados_por_tick}} \right\rceil \cdot \text{tick_segundos_reales} \quad (10)$$

El tiempo real observado se mide con la diferencia entre el timestamp real del primer snapshot de la corrida y el timestamp real del snapshot final. PC3 conserva esta metadata en su base de estado actual y la muestra en la pantalla final junto con el overhead:

$$\text{overhead} = T_{real_observado} - T_{teorico} \quad (11)$$

Esta métrica captura la diferencia práctica producida por procesamiento, comunicación ZeroMQ, persistencia SQLite, renderizado/consulta web, planificación del sistema operativo y carga acumulada de la simulación.

La snapshot operativa también incluye métricas agregadas del último tick (`created`, `eliminados` y `movidos`). PC3 guarda esas métricas en la metadata de su base de estado actual para que la interfaz pueda mostrar, sin consultar el histórico completo, cuántos vehículos fueron generados y cuántos fueron despawneados en el último avance de simulación.

9. Interacción entre Componentes

9.1. Flujo General del Sistema

El flujo operativo del sistema sigue esta cadena:

1. **PC0** genera vehículos y mantiene el estado base del tráfico.
2. **PC0** comunica el estado actualizado a PC3 (BD principal), a PC2 (réplica operativa) y a PC0 (base histórica).

3. **PC1** recibe las snapshots de **PC0** (véase la sección 9.2), ejecuta sensores y publica eventos a través del broker ZeroMQ.
4. **PC2** se suscribe a los eventos, calcula analítica por vía, agrupa scores por eje y determina fases semafóricas.
5. **PC3** permite monitorear, consultar y emitir indicaciones de control manual.

9.2. Snapshot Operativa

Una *snapshot operativa* puede entenderse como una foto del estado actual del sistema en un tick determinado. No representa una película completa del tráfico, sino un corte puntual del mundo simulado en un instante lógico. Esa snapshot contiene al menos el estado presente de intersecciones, vías y vehículos, junto con el tick actual y la marca temporal de simulación.

En términos prácticos, la snapshot sirve para que:

- **PC0 le diga a PC1**: “así está el mundo ahorita”.
- **PC1** use esa foto para calcular sensores.
- **PC2** y **PC3** guarden estado actual.
- **PC3** pueda resincronizarse desde **PC2** si volvió a levantarse.

Desde el punto de vista de arquitectura, la snapshot operativa es el contrato que evita que PC1, PC2 y PC3 tengan que reconstruir por su cuenta el estado del mapa. En lugar de duplicar la simulación, estos componentes consumen la foto autoritativa producida por PC0 y trabajan a partir de ella.

La cadencia de envío de la snapshot se controla por configuración. En la implementación actual, PC0 emite una primera snapshot en el primer tick de simulación y, a partir de ahí, vuelve a emitirla cada N ticks según el valor configurado en la siguiente clave:

`simulacion.intervalo_snapshot_ticks`

Esto permite balancear dos necesidades opuestas, es decir, por un lado, una observación más fina del sistema; por otro, menor volumen de mensajes y menor presión de persistencia.

Los endpoints ZeroMQ asociados a la propagación de estado también se definen por configuración. Durante pruebas locales pueden usarse direcciones de `localhost`, pero en despliegues sobre varios computadores esos endpoints deben reemplazarse por las IP o nombres reales de las máquinas participantes, sin necesidad de modificar la lógica del sistema.

9.3. Creación de Ambulancias

Cuando el usuario crea una ambulancia desde la interfaz de PC3:

1. PC3 valida localmente que el nodo BORDE-X-X exista y tenga *spawn* válido.
2. PC3 envía la solicitud por ZeroMQ a PC0, indicando el nodo de borde donde debe aparecer la ambulancia.

3. PC0 instancia la ambulancia como una entidad vehicular especial dentro de la simulación.
4. Mientras la ambulancia ocupa una vía, esa vía fuerza `score=1.0`; PC1 y PC2 conservan esa prioridad al publicar y recalcular sensores/comandos.
5. PC3 la visualiza con una representación diferenciada (por ejemplo, icono de sirena) respecto al tráfico normal.

La creación de la ambulancia recae en PC0 y la visualización/control en PC3. Es decir: **PC3 la pide por ZeroMQ y PC0 la crea realmente**. En la implementación operativa esto se modela con un canal dedicado donde el backend principal de PC3 emite una solicitud de ambulancia y PC0 la valida e inyecta en el motor de simulación.

9.4. Priorización Manual de Semáforos

Cuando el usuario fuerza un semáforo desde PC3:

1. PC3 envía la orden al servicio de analítica/control en PC2.
2. La orden indica qué intersección y qué eje priorizar, o si debe volver a modo automático.
3. PC2 ejecuta el forzado, suspendiendo la lógica automática en esa intersección hasta nueva orden.
4. Solo una intersección puede permanecer en modo manual a la vez; para cambiar de nodo, primero debe liberarse la anterior.

En la implementación actual, este forzado se realiza por ZeroMQ: el backend principal de PC3 emite la solicitud de control manual, PC2 la valida, verifica que no exista otra intersección retenida en manual y genera un comando semaforico manual que envía inmediatamente a PC0, donde se refleja en el siguiente ciclo operativo de la simulación. Cuando el usuario pulsa **Automatico**, PC2 libera la retención manual y restituye de inmediato el programa automático vigente para esa intersección. Si PC3 no está disponible, el backend de respaldo de PC2 no acepta nuevos controles manuales.

10. Inicialización del Sistema

10.1. Configuración Inicial

Se define un archivo, o un conjunto de archivos, de configuración compartidos que todos los componentes leen al arrancar. Esta configuración incluye al menos:

- Tamaño de la cuadrícula ($N \times M$).
- Intersecciones activas y nodos de borde.
- Número y tipo de sensores por arista o acceso instrumentado.
- Parámetros de semáforos (tiempo base y duración del ciclo).

- Pesos de ponderación de sensores y umbrales de analítica.
- Parámetros del reloj de simulación (hora inicio, hora fin, factor de velocidad).
- Parámetros de generación de vehículos (tasa de ingreso, velocidades).

Esta configuración centralizada permite que el arranque sea reproducible y que los parámetros se ajusten sin cambiar código.

10.2. Orden de Arranque

El orden de arranque acordado es:

1. **PC3**: levanta la base de datos principal y el backend primario.
2. **PC2**: levanta el servicio de analítica, el control semafórico y la réplica operativa de la base de datos.
3. **PC1**: levanta los sensores simulados y el broker ZeroMQ.
4. **PC0**: inicia la simulación autoritativa, la generación de vehículos y el almacenamiento histórico diario.

Este orden garantiza que los componentes consumidores y de persistencia estén disponibles antes de empezar a emitir eventos y a mover vehículos.

10.3. Prioridad de Implementación

Para la implementación incremental del proyecto se prioriza primero el núcleo operativo formado por **PC0**, **PC1** y **PC2**. En esta etapa se busca completar:

- simulación vehicular y estado base del mapa en PC0,
- sensado y broker en PC1,
- analítica, control semafórico y réplica operativa en PC2.

Esta decisión permitió estabilizar primero los elementos más delicados del sistema, es decir, el movimiento de vehículos por ticks, la medición correcta sobre aristas, el cálculo del score por vía, la emisión controlada de comandos semafóricos, la persistencia del estado actual y la propagación de la snapshot operativa entre computadores. De esta manera, las capas posteriores no se construyeron sobre un comportamiento todavía incierto o inconsistente.

Una vez estabilizado el flujo central PC0-PC1-PC2, se añade el backend principal de PC3 y el backend de respaldo reducido en PC2, enfocado solamente en continuidad y consulta del estado presente, dejando para una etapa posterior la visualización (frontend) del mapa.

11. Fallos y Continuidad Operativa

11.1. Caída de PC3

La falla principal considerada es la caída de PC3. Si PC3 falla, el backend principal deja de responder, pero el sistema no pierde el control operativo porque PC0, PC1 y PC2 continúan ejecutando la simulación, el sensado, la analítica y la réplica de estado. Desde el punto de vista del usuario, se pierden temporalmente:

- Visualización en tiempo real del mapa de la ciudad.
- Monitoreo interactivo y consultas desde la interfaz de usuario.
- Creación de nuevas ambulancias.
- Emisión de órdenes manuales de priorización semafórica.

11.2. Continuidad con PC2

Ante la caída de PC3, el sistema sigue operando con la réplica en PC2. Los componentes internos nunca dejan de trabajar y el respaldo puede exponer como máximo salud y consulta de estado actual para observación mínima.

Si PC3 se vuelve a levantar, el sistema no se queda indefinidamente en PC2. En ese caso, se sincroniza de nuevo la base de datos de PC3 con el estado actual que tiene PC2 y, al terminar esa actualización, la operación vuelve automáticamente a PC3 como base principal.

11.3. Backend Principal y Backend de Respaldo

El sistema distingue dos roles de backend:

- **Backend principal en PC3:** es el camino normal para consultas de estado, creación de ambulancias y control manual.
- **Backend de respaldo en PC2:** comparte el mismo protocolo de solicitudes, pero queda restringido a continuidad mínima, salud y estado actual.

La operación mínima de continuidad incluye también una consulta de **salud**. Esta consulta funciona como un *ping* del backend y permite saber qué servidor atendió la solicitud. Si responde PC3, el cliente sigue en el camino normal; si PC3 no responde y la petición cae a PC2, la respuesta identifica explícitamente al respaldo como backend activo.

El cliente durante el fallo intenta primero hablar con PC3; si no recibe respuesta, reintenta automáticamente la misma solicitud contra PC2. Esta decisión hace que el respaldo mantenga continuidad de protocolo, es decir, que pueda seguir respondiendo dentro del mismo esquema de solicitudes sin obligar al cliente a cambiar de interfaz o de contrato.

Sin embargo, compartir el mismo protocolo no significa compartir la misma autoridad operativa. El respaldo no ejecuta operaciones activas. Si recibe solicitudes como las siguientes:

`crear_ambulancia`

`control_manual`

responde explícitamente que la operación no está disponible en modo respaldo. Ese rechazo es deliberado y forma parte del diseño, para evitar que la continuidad mínima de PC2 se convierta en una segunda capa completa de control de usuario. Aunque el cliente durante el fallo podría reenviar esas solicitudes al respaldo, solo PC3 está autorizado para iniciarlas como operaciones activas de usuario.

11.4. Limitaciones Durante la Falla

Durante la falla de PC3, las siguientes funcionalidades quedan indisponibles:

- No se pueden crear nuevas ambulancias desde la interfaz.
- No se pueden emitir nuevas órdenes manuales de priorización semafórica.
- Se pierde la visualización y el monitoreo interactivo.

PC0 **no** participa en el cambio a un respaldo cuando ocurre una falla. El escenario de resiliencia es: PC3 cae y el sistema sigue operando con la réplica de PC2.

Durante el proceso de resincronización de PC3 pueden seguir ocurriendo cambios en el sistema mientras se copia el estado desde PC2. Por esta razón, se acepta que al volver a operar con PC3 pueda aparecer una pequeña inconsistencia temporal o una leve sensación de “retroceso en el tiempo” en la simulación. Esta limitación se considera aceptable dentro del alcance del proyecto.

12. Comparación de Rendimiento del Broker

12.1. Versión Base

Primera versión del broker en PC1 con una lógica simple y sin concurrencia interna. Recibe eventos y los reenvía a PC2 de manera secuencial. Esta es la línea base del experimento de rendimiento.

12.2. Versión Modificada con Hilos

Segunda versión del broker que introduce **hilos** para separar la recepción, el encolado y el reenvío de eventos en paralelo.

Métricas de comparación entre ambas versiones:

- Cantidad de eventos almacenados en la BD en una ventana de 2 minutos.
- Tiempo desde que el usuario solicita una acción hasta que el semáforo cambia.

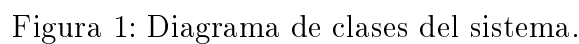
Los escenarios de prueba varían el número de sensores y el tiempo entre generación de mediciones.

13. Diagramas de Diseño

Esta sección recopila los diagramas UML que modelan la estructura y el comportamiento del sistema distribuido de gestión inteligente de tráfico urbano. Cada diagrama ofrece una perspectiva complementaria del diseño.

13.1. Diagrama de Clases

En la Figura 1 se presenta el diagrama de clases, que muestra las principales entidades del sistema (modelos de la ciudad, mensajes de comunicación y utilidades compartidas) junto con sus atributos y relaciones.



En la Figura 2 se presenta el diagrama de componentes, que describe los módulos lógicos de alto nivel del sistema y las interacciones principales entre ellos, sin hacer referencia directa al código fuente.

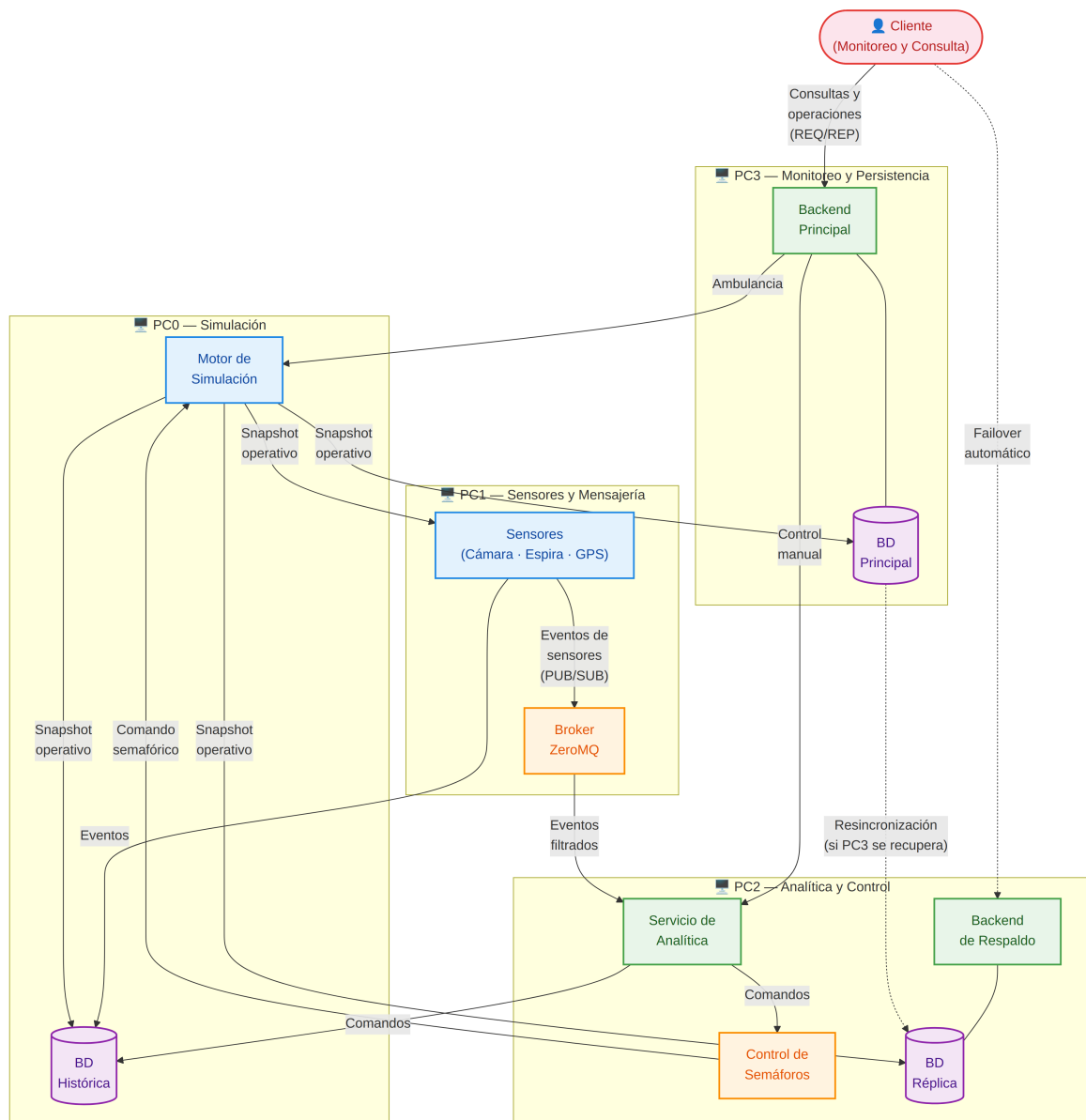


Figura 2: Diagrama de componentes del sistema.

13.3. Diagrama de Despliegue

En la Figura 3 se presenta el diagrama de despliegue, que ilustra la distribución física de los servicios en los cuatro computadores (PC0–PC3), los canales ZeroMQ que los conectan y las bases de datos asociadas.

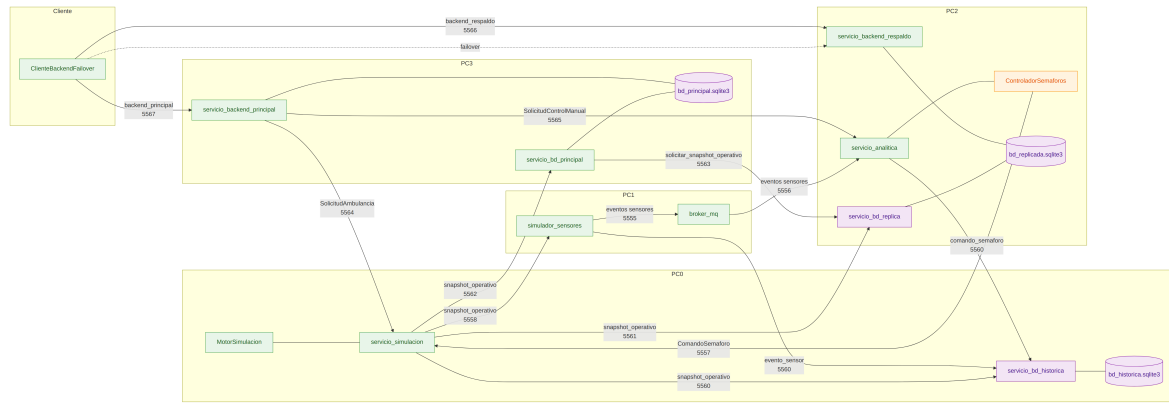


Figura 3: Diagrama de despliegue del sistema.

13.4. Diagrama de Secuencia

En la Figura 4 se presenta el diagrama de secuencia, que detalla el flujo temporal de mensajes durante un ciclo completo de simulación: desde la generación de un tick en PC0, pasando por la emisión de eventos de sensores y el cálculo analítico, hasta la aplicación de comandos semafóricos.

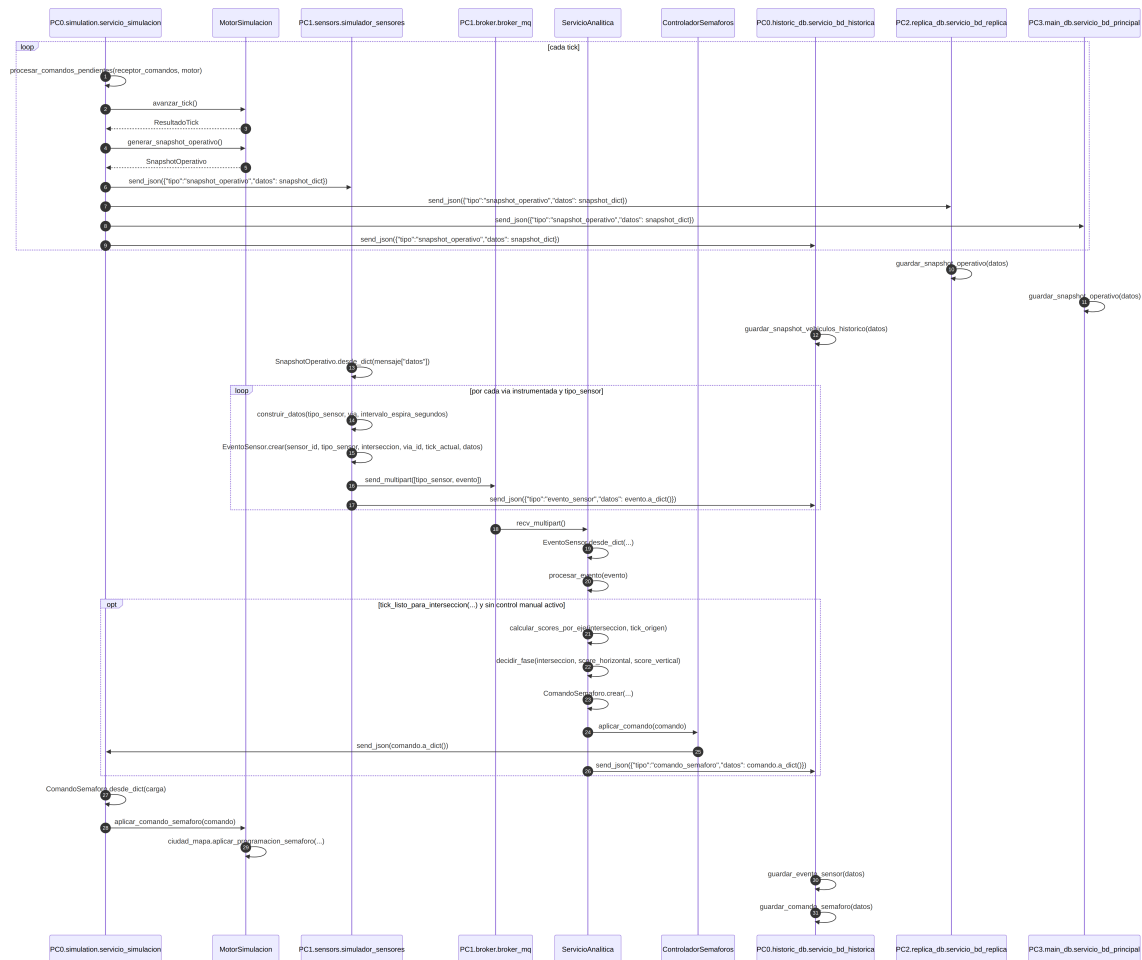


Figura 4: Diagrama de secuencia del sistema.

14. Protocolo de Pruebas

14.1. Objetivo

El presente protocolo define las pruebas necesarias para verificar y validar el correcto funcionamiento del sistema distribuido de gestión inteligente de tráfico urbano, con énfasis en las pruebas de desempeño exigidas por el enunciado del proyecto. El documento cubre pruebas funcionales, de integración, de fallos, de seguridad y de rendimiento comparativo entre la versión base del broker y la versión mejorada con hilos.

14.2. Alcance

El protocolo abarca la totalidad del sistema distribuido compuesto por cuatro computadores (PC0, PC1, PC2 y PC3), sus canales ZeroMQ, las tres bases de datos SQLite (histórica en PC0, réplica operativa en PC2 y principal en PC3), los tres tipos de sensores lógicos (cámara, espira inductiva y GPS), el servicio de analítica, el control de semáforos, el backend principal, el backend de respaldo y el mecanismo durante el fallo

de PC3. Se incluyen los escenarios mínimos de rendimiento definidos en la Tabla 1 del enunciado.

14.3. Supuestos

- Todas las pruebas se ejecutan sobre la misma configuración compartida definida en `config/system_config.json`, salvo las variaciones explícitas del escenario de rendimiento.
- Los endpoints ZeroMQ se configuran en `localhost` para pruebas locales y se reemplazan por direcciones IP reales cuando se ejecuta sobre tres o más máquinas físicas o virtuales.
- Las bases de datos SQLite se limpian antes de cada ejecución de prueba mediante el script de limpieza incluido en el directorio `scripts`, para evitar contaminación entre corridas según lo indicado en la sección 8.4.
- El orden de arranque respeta la secuencia documentada en la sección 10.2: PC3, PC2, PC1, PC0.
- La semilla pseudoaleatoria del motor de simulación y de la generación de vías se fija en el archivo de configuración para garantizar reproducibilidad.
- Se asume que el hardware de prueba permanece estable durante las mediciones y que no existen procesos externos que compitan significativamente por CPU o red.

14.4. Marco de Pruebas por Modelos del Sistema

14.4.1. Modelo Arquitectónico

El modelo arquitectónico del sistema organiza cuatro nodos (PC0, PC1, PC2, PC3) con responsabilidades definidas: PC0 como dueño autoritativo del mapa, los vehículos y el reloj lógico; PC1 como nodo de sensado y brokering; PC2 como nodo de analítica, control semafórico y réplica operativa; PC3 como nodo de persistencia principal, backend primario y punto de interacción con el usuario (véase la sección 1).

Aplicación al proyecto: las pruebas arquitectónicas verifican que cada nodo ejecuta exclusivamente las responsabilidades asignadas, que los canales ZeroMQ conectan los componentes según los endpoints definidos en la configuración (véase la sección 2.3), que la snapshot operativa fluye correctamente de PC0 a PC1 y que el estado actual se persiste tanto en PC3 como en PC2. Se valida que PC0 sea la única fuente de verdad para el estado del grafo, que PC1 no modifique el estado sino que lo observe, y que PC2 tome decisiones semafóricas sin duplicar la simulación.

14.4.2. Modelo de Interacción

El modelo de interacción describe los patrones de comunicación: PUB/SUB para la publicación de eventos de sensores desde PC1 hacia PC2 vía el broker; PUSH/PULL para el envío de comandos semafóricos desde PC2 hacia PC0, para la persistencia en bases de datos y para las solicitudes de ambulancias; REQ/REP para las consultas del usuario desde el cliente hacia el backend principal de PC3 (y durante el fallo hacia

PC2); y la snapshot operativa como contrato de estado compartido publicado por PC0 (véase las secciones 9.1 y 9.2).

Aplicación al proyecto: las pruebas de interacción verifican que los eventos de sensores se publican con tópicos diferenciados por tipo de sensor y que el broker los reenvía correctamente, que los comandos semafóricos llegan a PC0 y se aplican en el siguiente ciclo de simulación, que las solicitudes REQ/REP del usuario reciben respuesta dentro de un tiempo acotado, que las solicitudes de control manual llegan a PC2 y suspenden la lógica automática durante el período indicado (véase la sección 5.5), y que las solicitudes de ambulancia viajan de PC3 a PC0 y se materializan como vehículos de tipo `AMBULANCIA` en la simulación (véase la sección 9.3).

14.4.3. Modelo de Fallos

El modelo de fallos se centra en la caída de PC3 como escenario principal de falla. Si PC3 cae, el núcleo PC0-PC1-PC2 continúa operando y PC2 sostiene continuidad mínima con consulta de estado actual y salud, sin aceptar operaciones activas de usuario (véase las secciones 11.1 a 11.4). Cuando PC3 se reincorpora, solicita a PC2 el estado actual y resincroniza su base de datos.

Aplicación al proyecto: las pruebas de fallos simulan la caída de PC3 durante la operación normal y verifican que: (a) la simulación en PC0 no se detiene, (b) los sensores en PC1 siguen publicando, (c) la analítica en PC2 sigue emitiendo comandos, (d) el cliente de fallo redirige automáticamente las consultas al backend de respaldo de PC2, (e) las solicitudes de creación de ambulancia y de control manual son rechazadas explícitamente por el respaldo, y (f) al reiniciar PC3 se resincroniza con el estado actual de PC2 y la operación vuelve al camino principal.

14.4.4. Modelo de Seguridad

El modelo de seguridad se aplica desde la perspectiva de integridad y autoridad operativa. Solo PC0 puede crear y modificar el estado del mapa; solo PC3 puede iniciar operaciones activas de usuario; el backend de respaldo de PC2 rechaza deliberadamente operaciones que no le corresponden (véase la sección 11.3).

Aplicación al proyecto: las pruebas de seguridad verifican que el backend de respaldo de PC2 rechace con un mensaje explícito las solicitudes de creación de ambulancia y de control manual, que ningún componente distinto a PC0 pueda inyectar vehículos en la simulación, y que la configuración de endpoints ZeroMQ sea coherente entre todos los nodos para evitar conexiones cruzadas o no autorizadas.

14.5. Matriz de Casos de Prueba

14.5.1. Pruebas Funcionales

ID	Propósito	Pre condiciones	Pasos	Datos	Métrica	Criterio de éxito
PF-01	Verificar generación de eventos por los tres tipos de sensores	PC0, PC1 y PC2 iniciados; cuadrícula 2×2; 3 tipos de sensor activos	1. Iniciar PC0. 2. Iniciar PC1. 3. Observar logs de PC1.	Config. por defecto	Presencia de eventos camara , espiral y gps con tick_origen coherente	Se generan los 3 tipos de evento por cada arista instrumentada
PF-02	Verificar que el broker reenvía todos los eventos a PC2	PF-01 cumplida; PC2 iniciado	1. Observar logs de PC2. 2. Contrastar cantidad de eventos recibidos vs. publicados.	Config. por defecto	Razón eventos recibidos / enviados	Razón $\geq 0,99$
PF-03	Verificar cálculo correcto del <i>score</i> por vía	PC0, PC1 y PC2 en operación	1. Capturar notas parciales de un tick. 2. Calcular manualmente el <i>score</i> con los pesos configurados.	Notas del log; pesos de config.	$ \text{score}_{\text{esp}} - \text{score}_{\text{obs}} $	Diferencia $\leq 0,001$
PF-04	Verificar temporización semafórica según gap	PC2 en operación	1. Identificar comando con gap > 0 . 2. Verificar $T_{\text{verde}} = 15 + 15 \cdot \text{gap}$ y $T_{\text{opuesto}} = 30 - T_{\text{verde}}$.	Comando capturado del log	Adherencia a la fórmula	Tiempos coinciden exactamente
PF-05	Verificar deduplicación de comandos semafóricos	PC2 en operación; tráfico estable	1. Ejecutar ≥ 10 ticks. 2. Verificar ausencia de comandos consecutivos idénticos por intersección.	Logs de PC2	Pares duplicados consecutivos	Ningún par duplicado
PF-06	Verificar creación de ambulancia desde PC3	Sistema completo en operación	1. Enviar crear_ambulancia con nodo de borde válido a PC3. 2. Verificar log de PC0.	nodo_origen válido	Registro de ambulancia en PC0	Ambulancia aparece con tipo AMBULANCIA

ID	Propósito	Pre condiciones	Pasos	Datos	Métrica	Criterio de éxito
PF-07	Verificar control manual de semáforo	Sistema completo en operación	1. Enviar <code>control_manual</code> (INT-A1, VERTICAL). 2. Observar suspensión de lógica automática. 3. Enviar <code>AUTO</code> .	INT-A1, VERTICAL, AUTO	Log de aplicación y liberación	Forzado aplicado y liberado correctamente
PF-08	Verificar consulta de estado de intersección	PC3 y BD actualizadas con ≥ 1 snapshot	1. Enviar <code>estado_interseccion</code> . 2. Verificar campos de respuesta.	Intersección válida del grafo	Compleitud de campos	Fase activa, duración y tick coherentes
PF-09	Verificar consulta de resumen de estado	PC3 actualizado	1. Enviar <code>resumen_estado</code> . 2. Verificar estructura.	Ninguno adicional	Compleitud de campos	Información de intersecciones, vías y vehículos presente
PF-10	Verificar persistencia de snapshots en BD principal y réplica	≥ 10 ticks de operación	1. Consultar tablas de estado en PC3 y PC2. 2. Comparar datos.	Ninguno adicional	Consistencia entre BD principal y réplica	Datos de estado actual consistentes

14.5.2. Pruebas de Integración

ID	Propósito	Pre condiciones	Pasos	Datos	Métrica	Criterio de éxito
PI-01	Flujo extremo a extremo: vehículo $\rightarrow$ sensor $\rightarrow$ analítica $\rightarrow$ comando $\rightarrow$ simulación	Sistema completo arrancado en orden	1. Iniciar PC3, PC2, PC1, PC0. 2. Esperar 5 ticks. 3. Verificar cada eslabón de la cadena.	Config. por defecto	Presencia de cada eslabón	Flujo completo observable
PI-02	Propagación de snapshot de PC0 a PC1	PC0 y PC1 iniciados	1. Verificar recepción de snapshots en PC1. 2. Verificar coherencia de <code>tick_origen</code> en eventos generados.	Config. por defecto; <code>intervalo_snapshot_ticks</code> : 1	Coherencia de <code>tick_origen</code>	Ticks de eventos coinciden con tick del snapshot

ID	Propósito	Pre condiciones	Pasos	Datos	Métrica	Criterio de éxito
PI-03	Propagación consistente del reloj lógico	Sistema completo en operación	1. Capturar <code>tick_actual</code> en snapshots, <code>tick_origen</code> en eventos y en comandos. 2. Verificar ordenamiento causal.	Logs de PC0, PC1 y PC2	Monotonía creciente de ticks	Ticks estrictamente no decrecientes por componente
PI-04	Escritura asíncrona en las tres BD	≥ 20 ticks de operación	1. Consultar conteos de registros en BD de PC0, PC2 y PC3. 2. Verificar presencia.	Ninguno adicional	Registros en las 3 BD	PC0 acumula eventos y comandos; PC2 y PC3 reflejan estado actual

14.5.3. Pruebas de Fallos

ID	Propósito	Pre condiciones	Pasos	Datos	Métrica	Criterio de éxito
PFL-01	Continuidad operativa ante caída de PC3	Sistema estable ≥ 10 ticks	1. Detener PC3 abruptamente. 2. Esperar 10 ticks. 3. Verificar operación de PC0, PC1 y PC2.	Ninguno adicional	Continuidad de logs	Los tres nodos continúan sin interrupción
PFL-02	Fallo de PC3 hacia backend de respaldo	PC3 detenido; PC2 con backend respaldo activo	1. Enviar <code>salud</code> desde cliente. 2. Verificar respuesta del backend.	<code>tipo: salud</code>	Valor de <code>backend_atendio</code>	Respuesta proviene del respaldo
PFL-03	Rechazo de operaciones activas en respaldo	PC3 detenido	1. Enviar <code>crear_ambulancia</code> durante el fallo. 2. Enviar <code>control_manual</code> . 3. Verificar rechazo.	Solicitudes de operación activa	Campo <code>error</code> en respuesta	Ambas devuelven <code>operacion_no_disponible_en_respaldo</code>
PFL-04	Resincronización de PC3 al reincorporarse	PC3 detenido durante 15 ticks y luego reiniciado	1. Detener PC3. 2. Reiniciar PC3. 3. Verificar resincronización. 4. Verificar retorno al camino principal.	Ninguno adicional	<code>backend_atendio</code> tras resincronización	Consultas posteriores retornan a PC3 principal

ID	Propósito	Pre condiciones	Pasos	Datos	Métrica	Criterio de éxito
PFL-05	Analítica no interrumpida durante caída de PC3	PC3 detenido	1. Verificar que los comandos semafóricos se siguen emitiendo desde PC2 hacia PC0.	Logs de PC2 y PC0	Continuidad de comandos	Comandos llegan sin interrupción a PC0

14.5.4. Pruebas de Seguridad e Integridad

ID	Propósito	Pre condiciones	Pasos	Datos	Métrica	Criterio de éxito
PS-01	Solo PC0 inyecta vehículos	Sistema completo en operación	1. Verificar logs de creación de vehículos. 2. Confirmar que solo PC0 genera registros de creación.	Logs de todos los nodos	Fuente de creación	Todos los registros provienen de PC0
PS-02	Coherencia de endpoints ZeroMQ entre nodos	Archivos de config. cargados por cada nodo	1. Comparar endpoints de <code>connect</code> con endpoints de <code>bind</code> del nodo destino.	<code>config/system_config.json</code>	Corresponde <code>connect</code> / <code>bind</code>	Todos los pares son coherentes
PS-03	Respaldo no permite operaciones activas	PC2 con backend respaldo activo; PC3 detenido	1. Enviar <code>crear_ambulancia</code> y <code>control_mensual</code> directamente a PC2. 2. Verificar rechazo.	Solicitudes de operación activa	Presencia de error de rechazo	Ambas solicitudes rechazadas

14.5.5. Pruebas de Desempeño

ID	Propósito	Pre condiciones	Pasos	Datos	Métrica	Criterio de éxito
PD-01	Rendimiento BD en 2 min — Esc. 1, broker base	Sistema limpio; broker base; 1 sensor/tipo; eventos cada 10 s	1. Iniciar sistema. 2. Ejecutar 120 s. 3. Contar registros en BD histórica de PC0.	Intervalo: 10 s; 1 sensor/tipo	Registros / 120 s	Valor positivo y proporcional a aristas instrumentadas
PD-02	Rendimiento BD en 2 min — Esc. 1, broker multihilos	Igual a PD-01 con broker multihilos	Ídem PD-01	Ídem PD-01	Registros / 120 s	Comparar con PD-01

ID	Propósito	Pre condiciones	Pasos	Datos	Métrica	Criterio de éxito
PD-03	Rendimiento BD en 2 min — Esc. 2, broker base	Broker base; 2 sensores/tipo; eventos cada 2.5 s	Ítem PD-01 con configuración de Esc. 2	Intervalo: 2.5 s; 2 sensores/tipo	Registros / 120 s	Valor documentado
PD-04	Rendimiento BD en 2 min — Esc. 2, broker multihilos	Igual a PD-03 con broker multihilos	Ítem PD-03	Ítem PD-03	Registros / 120 s	Comparar con PD-03
PD-05	Latencia usuario → cambio semáforo — Esc. 1, broker base	Sistema estable; broker base; Esc. 1	1. Registrar t_0 al enviar <code>control_manual</code> 1. 2. Registrar t_1 cuando PC0 aplica el comando. 3. Calcular $\Delta t = t_1 - t_0$.	<code>control_manual</code> con intersección válida	Δt (ms)	$\Delta t < 2000$ ms en local
PD-06	Latencia usuario → cambio semáforo — Esc. 1, broker multihilos	Igual a PD-05 con broker multihilos	Ídem PD-05	Ídem PD-05	Δt (ms)	Comparar con PD-05
PD-07	Latencia usuario → cambio semáforo — Esc. 2, broker base	Sistema estable; broker base; Esc. 2	Ídem PD-05	Esc. 2	Δt (ms)	Valor documentado
PD-08	Latencia usuario → cambio semáforo — Esc. 2, broker multihilos	Igual a PD-07 con broker multihilos	Ídem PD-07	Ídem PD-07	Δt (ms)	Comparar con PD-07
PD-09	Latencia de procesamiento analítico por tick	PC2 en operación	1. Registrar timestamp de recepción del último evento del tick. 2. Registrar timestamp de emisión del comando. 3. Calcular diferencia.	Logs de PC2	Δt recepción–emisión	Métrica complementaria documentada

ID	Propósito	Pre condiciones	Pasos	Datos	Métrica	Criterio de éxito
PD-10	Latencia de propagación de snapshot PC0→PC1	PC0 y PC1 en operación	1. Registrar timestamp de emisión en PC0. 2. Registrar timestamp de recepción en PC1. 3. Calcular diferencia.	Logs de PC0 y PC1	Δt de propagación	Métrica complementaria documentada
PD-11	Rendimiento BD en 2 min — Esc. 3, broker base	Sistema limpio; broker base; 3 sensores/tipo; eventos cada 1 s	Ídem PD-01 con configuración de Esc. 3	Intervalo: 1 s; 3 sensores/tipo	Registros / 120 s	Valor documentado
PD-12	Rendimiento BD en 2 min — Esc. 3, broker multihilos	Igual a PD-11 con broker multihilos	Ídem PD-11	Ídem PD-11	Registros / 120 s	Comparar con PD-11
PD-13	Latencia usuario → cambio semáforo — Esc. 3, broker base	Sistema estable; broker base; Esc. 3	Ídem PD-05	Esc. 3	Δt (ms)	Valor documentado
PD-14	Latencia usuario → cambio semáforo — Esc. 3, broker multihilos	Igual a PD-13 con broker multihilos	Ídem PD-13	Ídem PD-13	Δt (ms)	Comparar con PD-13

14.6. Procedimiento de Ejecución y Repetición Experimental

14.6.1. Preparación del entorno

1. Verificar que todas las dependencias del archivo `requirements.txt` estén instaladas en cada máquina participante.
2. Verificar que el archivo `config/system_config.json` es idéntico en todos los nodos.
3. Ejecutar el script de limpieza de bases de datos (`scripts/limpiar_bases.sh`).
4. Verificar que no existen procesos residuales de ejecuciones anteriores.

14.6.2. Ejecución de una corrida

1. Arrancar los nodos en el orden documentado: PC3, PC2, PC1, PC0.
2. Esperar al menos 2 minutos reales para las pruebas de rendimiento, o el número de ticks necesario para las pruebas funcionales.

3. Registrar el inicio y el fin de la ventana de medición.
4. Al finalizar la ventana, detener los nodos en orden inverso: PC0, PC1, PC2, PC3.

14.6.3. Repetición experimental

Cada prueba de desempeño se repite un mínimo de **5 veces** con la misma configuración y la misma semilla de simulación. Se reportan:

- Media aritmética de la métrica.
- Desviación estándar.
- Valor mínimo y máximo.
- Mediana.

La primera corrida de cada serie se descarta como corrida de calentamiento para mitigar efectos de caché y estabilización de conexiones ZeroMQ. En consecuencia, se ejecutan 6 corridas y se analizan las 5 últimas.

14.6.4. Configuración de escenarios

Parámetro	Escenario 1	Escenario 2	Escenario 3
Sensores por tipo por arista	1	2	3
Intervalo de publicación	10 s	2.5 s	1 s
Tipo de broker	Base / Multihilos	Base / Multihilos	Base / Multihilos
Duración de ventana de medición	120 s	120 s	120 s
Cuadrícula	Según configuración	Según configuración	Según configuración
Semilla de simulación	Fija	Fija	Fija

La combinación genera 6 tratamientos experimentales: (Escenario 1, Base), (Escenario 1, Multihilos), (Escenario 2, Base), (Escenario 2, Multihilos), (Escenario 3, Base), (Escenario 3, Multihilos).

14.7. Tratamiento de Fallas: Caída de PC3 y Continuidad con Réplica PC2

14.7.1. Protocolo de prueba de caída

1. El sistema se ejecuta en estado estable durante al menos 20 ticks.
2. Se detiene abruptamente el proceso de PC3.
3. Se verifica en los logs de PC0, PC1 y PC2 que no hay interrupciones.
4. Se envían consultas desde el cliente durante un fallo y se verifica que las respuestas provienen del backend de respaldo.
5. Se intentan operaciones activas (crear ambulancia, control manual) y se verifica su rechazo.

6. Se reinicia PC3.
7. Se verifica la resincronización consultando la base de datos de PC3 y comparándola con el estado actual de PC2.
8. Se envían nuevas consultas y se verifica que el backend principal de PC3 vuelve a atender.

14.7.2. Criterios de éxito

- La simulación no se detiene en ningún momento.
- El fallo de PC3 es transparente y automático.
- Las operaciones activas son rechazadas por el respaldo con mensajes explícitos.
- Tras la resincronización, PC3 atiende nuevamente con datos consistentes.

14.8. Riesgos a la Validez de Resultados

14.8.1. Amenazas a la validez interna

- **Carga del sistema operativo:** procesos no controlados en las máquinas de prueba pueden introducir variabilidad en las mediciones de latencia y rendimiento. Se mitiga ejecutando las pruebas en un entorno limpio con carga mínima.
- **Sincronización de relojes:** si las pruebas se ejecutan en máquinas distintas, las diferencias de reloj pueden distorsionar las mediciones de latencia extremo a extremo. Se mitiga con NTP o, preferiblemente, ejecutando mediciones de latencia en la misma máquina.
- **Efecto de calentamiento:** las primeras corridas pueden mostrar latencias mayores por inicialización de conexiones ZeroMQ y caché de SQLite. Se mitiga descartando la primera corrida de cada serie.
- **Acumulación en bases de datos:** si las bases no se limpian entre corridas, el rendimiento de las consultas SQL puede degradarse. Se mitiga con limpieza obligatoria entre corridas.

14.8.2. Amenazas a la validez externa

- **Escala reducida de la cuadrícula:** las pruebas con cuadrículas pequeñas (por ejemplo, 2×2) pueden no reflejar el comportamiento del sistema en cuadrículas mayores. Se recomienda documentar los resultados junto con el tamaño de la cuadrícula utilizada y, si es posible, repetir con una cuadrícula de mayor tamaño.
- **Red local vs. red real:** las pruebas en `localhost` eliminan la latencia de red. Los resultados no son extrapolables directamente a un despliegue en red real sin ajustar las expectativas.
- **Número limitado de repeticiones:** con 5 corridas efectivas la potencia estadística es baja. Se recomienda aumentar a 10 corridas si el tiempo lo permite.

14.8.3. Amenazas a la validez de constructo

- **rendimiento como proxy de capacidad:** la cantidad de registros en BD en 2 minutos depende no solo del broker sino también de la velocidad de escritura de SQLite. Se recomienda diferenciar ambos factores si es posible.
- **Latencia extremo a extremo como métrica compuesta:** la latencia medida incluye transmisión ZeroMQ, procesamiento analítico y aplicación del comando, y no permite aislar cuellos de botella sin instrumentación adicional. Se mitiga con las métricas complementarias PD-09 y PD-10.

14.9. Resultados Esperados y Formato de Reporte

Los resultados de las pruebas de desempeño se presentarán en:

- **Tablas comparativas** con los 6 tratamientos (3 escenarios $\times$ 2 versiones de broker), mostrando media, desviación estándar, mínimo, máximo y mediana para cada métrica.
- **Gráficos de barras o de caja** de rendimiento y latencia por tratamiento.
- **Análisis comparativo** que responda la pregunta del enunciado: ¿qué diseño es más escalable? La justificación se basará en las tendencias observadas al pasar del escenario 1 al escenario 2 y al escenario 3, y en la diferencia relativa entre broker base y broker multihilos.
- **Especificaciones de hardware y software** de las máquinas utilizadas (procesador, RAM, sistema operativo, versión de Python, versión de ZeroMQ).

14.10. Medidas de rendimiento

Escenario	Qué significa	Diseño solicitado en el proyecto	Diseño multihilos
Sensores cada 10 s. Spawn rate normal	Pocos eventos por sensor y tráfico normal. Ejemplo: los sensores reportan lentamente y la simulación genera vehículos a la tasa usual.	Se mide: eventos en BD en 2 min; tiempo de cambio del semáforo. Esperado: desempeño estable y baja latencia.	Se mide: eventos en BD en 2 min; tiempo de cambio del semáforo. Esperado: comportamiento similar o levemente mejor.
Sensores cada 2.5 s. Spawn rate normal	Más eventos de sensores, pero con tráfico normal. Ejemplo: los carros aparecen al ritmo usual, pero los sensores informan más seguido.	Se mide: eventos en BD en 2 min; tiempo de cambio del semáforo. Esperado: mayor carga sobre el broker y aumento moderado de latencia.	Se mide: eventos en BD en 2 min; tiempo de cambio del semáforo. Esperado: mejor manejo del aumento de eventos.

Escenario	Qué significa	Diseño solicitado en el proyecto	Diseño multihilos
Sensores cada 1 s. Spawn rate normal	Alta frecuencia de publicación de sensores con tráfico normal. Ejemplo: los sensores reportan casi continuamente.	Se mide: eventos en BD en 2 min; tiempo de cambio del semáforo. Esperado: mayor presión sobre el broker y mayor latencia.	Se mide: eventos en BD en 2 min; tiempo de cambio del semáforo. Esperado: menor degradación que la versión base.
Sensores cada 10 s. Spawn rate alto	Poca frecuencia de sensores, pero muchos vehículos en simulación. Ejemplo: aparecen más carros de lo normal, aunque los sensores reportan lentamente.	Se mide: eventos en BD en 2 min; tiempo de cambio del semáforo. Esperado: desempeño aceptable con más presión indirecta por tráfico.	Se mide: eventos en BD en 2 min; tiempo de cambio del semáforo. Esperado: comportamiento más estable que la versión base.
Sensores cada 2.5 s. Spawn rate alto	Frecuencia media de sensores y tráfico alto. Ejemplo: muchos carros y sensores reportando con bastante frecuencia.	Se mide: eventos en BD en 2 min; tiempo de cambio del semáforo. Esperado: carga exigente y latencia visible.	Se mide: eventos en BD en 2 min; tiempo de cambio del semáforo. Esperado: mejor tolerancia a carga concurrente.
Sensores cada 1 s. Spawn rate alto	Máxima exigencia del experimento. Ejemplo: muchos carros y sensores enviando datos todo el tiempo.	Se mide: eventos en BD en 2 min; tiempo de cambio del semáforo. Esperado: mayor probabilidad de congestión del broker y alta latencia.	Se mide: eventos en BD en 2 min; tiempo de cambio del semáforo. Esperado: mejor escalabilidad relativa bajo máxima carga.